

AI FITNESS VOICE ASSISTANT

Submitted in partial fulfilment of the requirements of the degree

BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

By

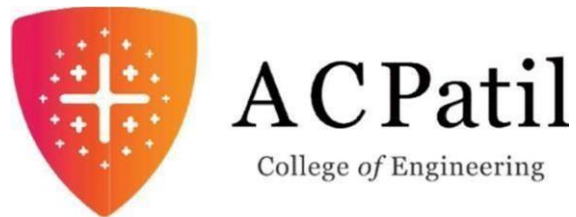
Altamash Chougale 221041042 / 05

Atharva Harane 221041026 / 15

Darsh Kamble 221041036 / 23

Guide

Prof. M. A. DALAL



Department of Computer Engineering

A.C. Patil College of Engineering

Kharghar, Navi Mumbai

University of Mumbai

(AY 2025-26)



**Jawahar Education Society's
A. C. Patil College of Engineering, Kharghar
Department of Computer Engineering
Academic Year 2025-26**

CERTIFICATE

This is to certify that the Mini Project (CSM 501-2A) entitled “**AI FITNESS VOICE ASSISTANT**” have been successfully completed by the following students:

Roll numbers	Name
05	Altamash Chougale
15	Atharva Harane
23	Darsh Kamble

In partial fulfillment of Natural Language Processing, (Semester-VII), submitted to the University of Mumbai for the award of the degree of “**Bachelor of Engineering**” in “**Computer Engineering**” during the academic year (2025-26).

Prof. Madhuri Dalal
Internal Guide
Subject In-charge

TABLE OF CONTENTS

Chapter	Title		Page No.
1	Introduction		3
2	Literature survey		6
3	Research Methodology		8
	3.1	Architecture/ Framework	8
	3.2	Algorithm and Process Design	9
	3.3	Details of Hardware & Software	12
	3.4	Code	12
	3.5	Experimental results	15
4	Conclusion and Future work		18
5	References		19

ABSTRACT

This project presents the design and implementation of a custom web-based interface for an **AI fitness voice assistant**. The primary objective is to create a feature-rich, user-friendly, and highly customizable client that integrates seamlessly with the Vapi AI platform. The system leverages standard web technologies—HTML, CSS, and JavaScript—to build a responsive user interface that allows for real-time, bidirectional communication with a voice AI. Key features include starting and stopping voice sessions, real-time transcription of the conversation, the ability to select different AI voices, and a text-based input method for sending messages. The application demonstrates how to effectively use the Vapi SDK to handle call lifecycle events, speech detection, and message exchanges. The final result is a robust and interactive web application that serves as a powerful front-end for advanced conversational AI, showcasing a practical approach to integrating sophisticated voice capabilities into any website or web application.

ACKNOWLEDGEMENTS

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people whose cooperation made it possible. We are grateful to our guide **Prof. M. A. Dalal** for the guidance, inspiration and constructive suggestions that were helpful to us in the preparation of this project. We would also like to thank the developers and community behind the Vapi AI platform for providing the powerful tools and documentation that formed the backbone of our assistant's capabilities.

1.INTRODUCTION

1.1 Introduction

- In recent years, conversational Artificial Intelligence (AI) has progressed significantly, evolving from basic rule-based chatbots into highly advanced voice assistants. These modern systems can understand complex context, perform multi-step tasks, and engage in natural, human-like dialogue. This evolution is largely driven by breakthroughs in deep learning, particularly the advent of powerful Large Language Models (LLMs), which have drastically improved Natural Language Processing (NLP) capabilities. As a result, these technologies are increasingly being adopted across diverse domains such as customer service, personal assistance, healthcare, and accessibility tools, where they enhance user interaction and efficiency.
- This rapid advancement has also reshaped user expectations; individuals now anticipate interactions with digital systems to be seamless, personalized, and context-aware. A simple, functional interface is no longer sufficient. Users expect an experience that is deeply integrated into the application they are using, reflecting a high degree of polish and intelligence. This project aims to meet these modern expectations by designing and developing a custom front-end application that provides an interactive and dedicated interface for communication with a sophisticated voice assistant, demonstrating a practical approach to building next-generation user experiences.

1.2 Motivation

- Although many platforms provide pre-built widgets for deploying voice assistants, they are often limited in terms of flexibility and integration. Such generic solutions may not blend well with a company's brand identity, nor do they always deliver the specific user experience that businesses desire. The motivation for this project arises from the need to create a custom solution that overcomes these constraints by offering full control over the interface, interaction flow, and overall design.
- A significant limitation of off-the-shelf widgets is their inability to handle custom business logic or specialized "function calls" in a visually integrated way. For instance, a healthcare assistant might need to fetch patient data from a secure API and display it within the application. A generic widget cannot be easily modified to support such specific backend integrations and corresponding UI updates. By building the client application from scratch, this project demonstrates how to create a front-end that can listen for and visually represent these custom events, providing a richer and more functional user experience.

1.3 Problem Statement and Objectives

To design and develop a fully customized web client for advanced voice AI that overcomes the limitations of rigid, off-the-shelf widgets, enabling seamless integration with backend AI services, providing complete flexibility in user interface and interaction flow, and delivering a highly responsive, intuitive, and user-centric experience. The project aims to establish a clear architectural framework and implementation approach that allows developers and businesses to deploy a scalable, reliable, and adaptable solution tailored to unique application requirements, while enhancing usability, engagement, and overall system performance.

Objectives:

- To design and develop a responsive and visually appealing web interface for an AI voice assistant using HTML, CSS, and JavaScript, ensuring compatibility across different devices and screen sizes, and providing an intuitive platform for user interaction.
- To integrate the VAPI AI SDK to manage seamless communication between the web client and the voice assistant, enabling reliable connectivity, real-time processing of audio inputs, and proper handling of backend requests and responses.
- To implement the core functionalities of the voice assistant, including starting and stopping conversations, displaying a live transcript of user inputs and assistant responses, and supporting text-based messaging for users who prefer typing, ensuring a fully interactive experience.
- To provide users with options to personalize and customize their experience, such as selecting the assistant's voice, adjusting speech speed, and choosing language preferences, thereby enhancing accessibility and engagement.
- To ensure a stable, responsive, and intuitive user experience by handling various interaction states effectively (e.g., ready, listening, processing, or error), providing appropriate visual and audio feedback, and maintaining smooth operation under different usage conditions.

1.4 Organization of the Report

Chapter 1:

This Chapter includes initiation of this project where we will analyze Problem, Project Motivation & Objective. This will be further categorized as:

- 1. Introduction**
- 2. Motivation**
- 3. Problem Statement**
- 4. Objective**

Chapter 2:

This Chapter is all about the analysis of existing systems, the problems and flaws of this system, the reference we used to make this project and further reading.

This will be further categorized as:

- 1. Survey of existing system**
- 2. Limitation Of existing system**

Chapter 3:

Here, we analyze the architecture and the implementation of our software, it also looks at the possible obstacles and the actual obstacles faced along with our test runs and conclusion.

This will be further categorized into:

- 1. Introduction**
- 2. Architecture/framework**
- 3. Algorithm of application**
- 4. Details on software and hardware used**
- 5. Test criteria**
- 6. Test result**
- 7. Conclusion**
- 8. Further work**

2.LITERATURE SURVEY

2.1 Survey of existing system

- Embedded widgets are the most commonly used solutions for integrating voice AI into websites. Platforms like VAPI AI, Voiceflow, and others provide a small JavaScript snippet that can be added to a webpage. Once implemented, the snippet typically renders a floating button or a mini chat interface for user interaction. These widgets are popular because they are quick to deploy and require minimal technical expertise, making it easy for websites to add voice assistant functionality without major development effort.
- However, these embedded widgets offer limited customization. Developers can generally only adjust superficial visual elements, such as the color scheme, size, or placement of the button. The deeper user interface and interaction flow cannot be restructured, which restricts the ability to align the voice assistant with the specific design and functionality needs of the application.
- The primary limitation of full-scale applications is that they are closed-source. Developers cannot fully embed these assistants into third-party websites, and customization of the interface or interaction logic is highly restricted. As a result, organizations seeking a branded or deeply integrated user experience cannot leverage these systems effectively for their own platforms.

2.2 Limitation of existing system

- The main limitation of existing embedded widgets is their “one-size-fits-all” design. While they are convenient for rapid deployment, they do not allow developers to rebuild the user interface or deeply integrate voice interactions into the application. This creates a significant constraint when trying to provide a fully customized and seamless user experience.
- Full-scale applications, on the other hand, provide advanced AI functionality but are closed-source and offer very limited flexibility for integration into third-party websites. The combination of these limitations reveals a clear gap: there is no existing standard or guide for building a fully custom voice AI client that provides complete control over design, interaction flow, and backend communication.
- Addressing this gap motivates the development of a custom client that can leverage a voice AI SDK to deliver a fully flexible, user-centric, and highly interactive web interface. Such a system would allow developers to implement unique branding, interaction design, and workflow, providing an optimal experience for end users.

2.3 Mini Project Contribution

❑ Requirement Analysis & Backend Integration

Name: Darsh Kamble

- Researched and integrated the VAPI AI platform using the SDK.
- Configured API keys and assistant IDs to establish a stable backend connection.
- Defined the event-driven architecture to handle frontend reactions to VAPI events like call-start, messages, and transcripts.

❑ Core Logic & State Management

Name: Altamash Chougale

- Developed client-side JavaScript logic to manage the assistant's lifecycle.
- Engineered state management to track application status (e.g., ready, listening, in-call) and update the UI.
- Implemented event listeners to handle real-time VAPI communication, including speech, transcripts, and assistant responses.

❑ Frontend UI/UX Design & Development

Name: Atharva Harane

- Designed a responsive, user-friendly frontend using HTML and CSS.
- Built interactive components, including the transcript area, voice selection, and control buttons.
- Enhanced user experience with visual feedback like loading indicators, button states, and smooth animations.

❑ Testing & Quality Assurance

All Members

- Conducted end-to-end testing to verify integration between UI and backend.
- Performed functional testing of all features across different browsers.
- Conducted User Acceptance Testing (UAT) for usability and responsiveness.
- Collaboratively fixed bugs in event handling, UI rendering, and state synchronization to ensure a stable application.

3. RESEARCH METHODOLOGY

2.4 Introduction:

The proposed system is a client-side web application that serves as a fully custom front-end for the Vapi AI voice assistant platform. Built entirely with front-end technologies, it is serverless, lightweight, and can be easily deployed on any standard web host without complex backend setup. The system follows a modular design, maintaining a clear separation between the user interface (HTML/CSS) and the underlying logic (JavaScript), which ensures easier maintenance, scalability, and the ability to incorporate future enhancements or new features seamlessly.

2.5 Architecture/Framework

The system follows simple, event-driven client-side architecture.

UI Layer (HTML/CSS): Defines the structure and style of the application, including the conversation transcript area, control buttons, and status indicators.

Logic Layer (JavaScript): This is the core of the application. It is responsible for:

- Initializing the Vapi AI SDK with the provided API key and Assistant ID.
- Managing the application's state (e.g., `isListening`, `vapiInstance`).
- Handling user interactions (e.g., button clicks, text input).
- Subscribing to and reacting to events from the Vapi SDK (e.g., `call-start`, `transcript`, `message`, and `error`).

Service Layer (Vapi AI SDK): An external JavaScript library that is loaded dynamically. It handles the complex underlying processes:

- Establishing a WebSocket connection to the Vapi servers.
- Capturing microphone audio and streaming it.
- Receiving audio from the server and playing it back.
- Emitting events that the Logic Layer can listen to.

2.6 Algorithm and Process Design

The core process flow is event-driven and can be described as follows:

1. Initialization:

- On page load (DOMContentLoaded), the initializeVapi function is called.
- A <script> tag pointing to the Vapi SDK is dynamically added to the document.
- Once the script loads, vapiSDK.run() is invoked to create an instance of the Vapi client.
- Event listeners for all Vapi events (e.g., call-start, message, transcript) are registered via setupVapiListeners.
- The UI is updated to a 'Ready' state, indicating the system is fully initialized and awaiting user interaction.

2. Starting a Conversation:

- The user clicks the "Start" button.
- The click event triggers vapiInstance.start(), which handles microphone permissions and establishes the call.
- The call-start event is received, and the UI updates to show 'Call in Progress'.

3. During the Conversation:

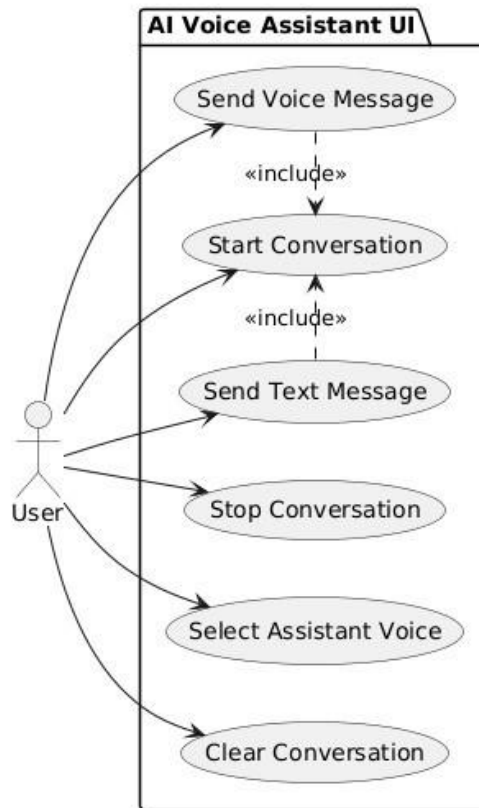
- **User Speaks:** The SDK detects speech via the speech-start event; the UI shows a 'Listening' status. When speaking stops (speech-end), the audio is sent for processing. A transcript event updates the UI with the transcribed text.
- **Assistant Responds:** The assistant processes the user input and sends back a response, triggering a message event. The UI displays the text, and the SDK plays the assistant's audio.
- **Text Input:** Users can type a message and click "Send", which calls vapiInstance.send(). This triggers the same processing workflow as spoken input.

4. Ending a Conversation:

- The user clicks the "Stop" button.
- The click event calls vapiInstance.stop(), ending the call.
- The call-end event is received, the connection is terminated, and the UI resets to the 'Ready' state, allowing a new interaction to begin.

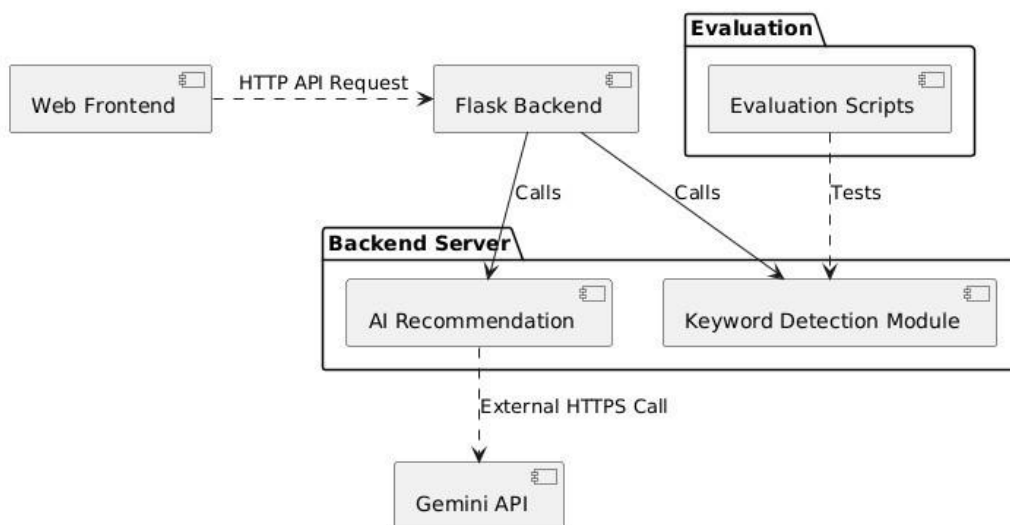
2.6.1 USE CASE DIAGRAM -

AI Voice Assistant Use Case Diagram

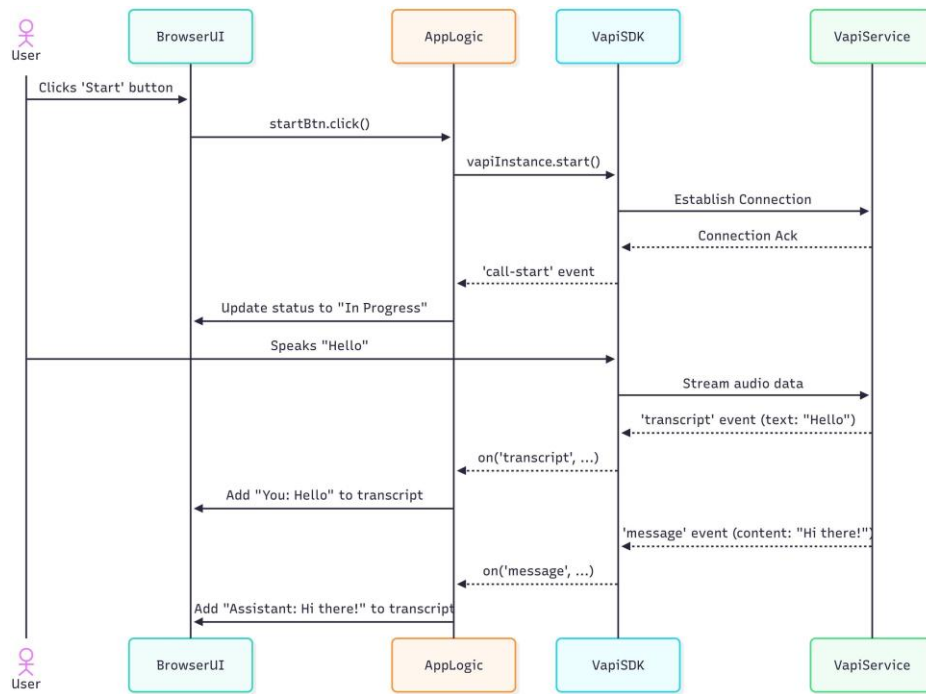


2.6.2 COMPONENT DIAGRAM -

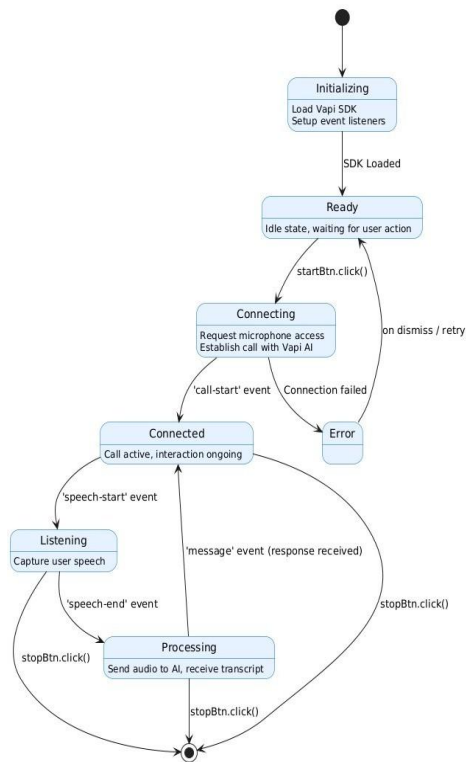
System Components



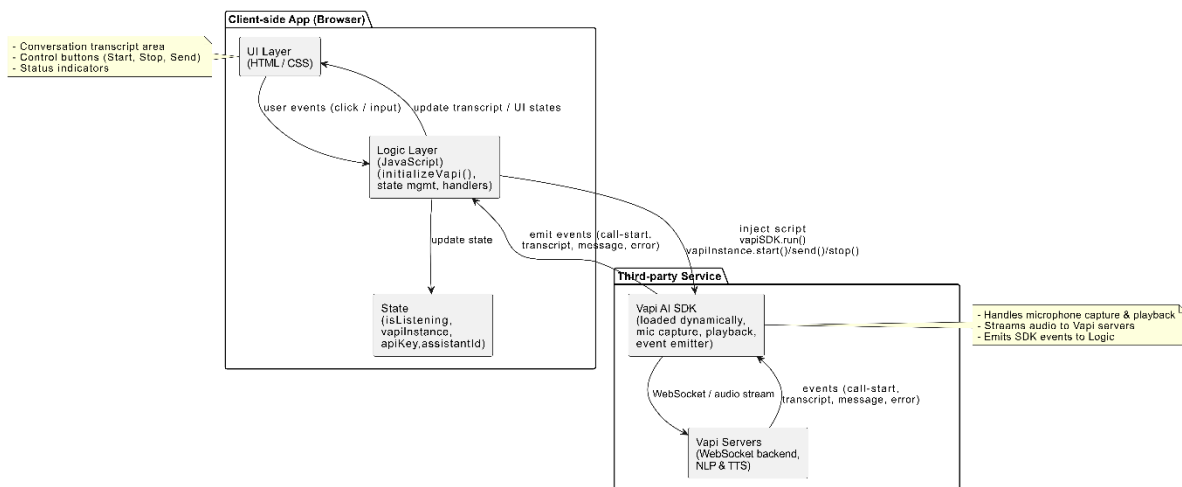
2.6.3 SEQUENCE DIAGRAM -



2.6.4 STATE TRANSITION DIAGRAM -



2.6.5 SYSTEM ARCHITECTURE –



2.7 Details of Hardware and Software

- **Hardware:**
 - A standard personal computer or mobile device with a modern web browser.
 - A functional microphone for voice input.
 - Speakers or headphones for audio output.
- **Software:**
 - Frontend: HTML5, CSS3, Vanilla JavaScript (ES6+).
 - AI Service Platform: Vapi AI, which orchestrates the backend services.
 - Underlying AI & Voice Technologies (via Vapi):
 - AI Model: The assistant logic is powered by a Large Language Model like GPT-4.
 - Text-to-Speech (TTS): The project integrates with high-quality TTS providers like ElevenLabs and Deepgram for generating the assistant's voice, as configured in the UI.
 - Speech Recognition: Vapi's service handles the real-time conversion of the user's speech into text for the AI model to process.
 - Development & Runtime Environment:
 - Web Browser: Google Chrome, Mozilla Firefox, or any modern browser with support for WebRTC and the Web Audio API.
 - Code Editor: Visual Studio Code.

2.8 Code

1) index.html and index.css

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

<title>AI Voice Assistant</title>
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/font-awesome@6.4.0/css/all.min.css">
</head>
<body>
  <div class="container">
    <header>
      <div class="logo">
        <div class="logo-icon"><i class="fas fa-microphone-alt"></i></div>
        <h1>AI Voice Assistant</h1>
      </div>
      <div class="status-indicator">
        <div class="status-dot" id="statusDot"></div>
        <span id="statusText">Ready to start</span>
      </div>
    </header>

    <div class="main-content">
      <div class="controls">
        <div class="control-group">
          <h2><i class="fas fa-voice"></i> Voice Options</h2>
          <div class="voice-options">
            <div class="voice-option active" data-voice="sarah">
              <div class="voice-avatar">👩</div>
              <div class="voice-info">
                <div class="voice-name">Sarah</div>
                <div class="voice-provider">11Labs</div>
              </div>
            </div>
            <div class="voice-option" data-voice="david">
              <div class="voice-avatar">👨</div>
              <div class="voice-info">
                <div class="voice-name">David</div>
                <div class="voice-provider">11Labs</div>
              </div>
            </div>
            <div class="voice-option" data-voice="emily">
              <div class="voice-avatar">👧</div>
              <div class="voice-info">
                <div class="voice-name">Emily</div>
                <div class="voice-provider">Deepgram</div>
              </div>
            </div>
          </div>
        </div>

        <div class="control-group">
          <h2><i class="fas fa-bolt"></i> Quick Actions</h2>
          <div class="action-buttons">
            <button id="startBtn" class="btn btn-primary">
              <i class="fas fa-microphone"></i> <span>Start Voice Assistant</span>
            </button>
            <button id="stopBtn" class="btn btn-secondary" disabled>
              <i class="fas fa-stop"></i> <span>Stop Assistant</span>
            </button>
            <button id="clearBtn" class="btn btn-secondary">
              <i class="fas fa-trash-alt"></i> <span>Clear Conversation</span>
            </button>
          </div>
        </div>
      </div>
    </div>
  </div>

```

```

    </div>
  </div>

  <div class="control-group">
    <h2><i class="fas fa-info-circle"></i> Assistant Info</h2>
    <p style="line-height: 1.6; font-size: 14px;">
      <!-- This assistant uses GPT-4 to answer your questions. -->
      You can ask about current events, technology, or general knowledge.
    </p>
  </div>
</div>

<div class="conversation">
  <div class="transcript" id="transcript">
    <div class="message assistant-message">
      <div class="message-header"><i class="fas fa-robot"></i> Assistant</div>
      <div class="message-content">Hello! I'm your assistant. Click the Start button to begin our conversation.</div>
    </div>
  </div>

  <div class="typing-indicator" id="typingIndicator">
    <span></span>
    <span></span>
    <span></span>
  </div>

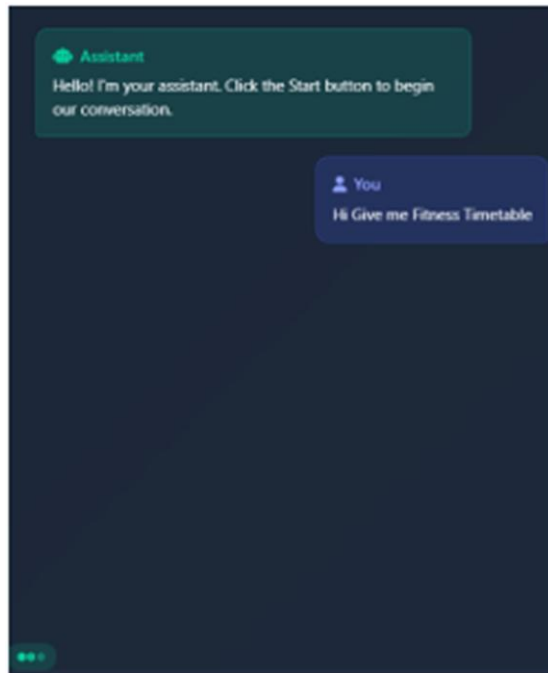
  <div class="input-area">
    <input type="text" id="userInput" placeholder="Type your message here...">
    <button id="sendBtn">
      <i class="fas fa-paper-plane"></i>
    </button>
  </div>
</div>
</div>
</div>
</body>
</html>

```

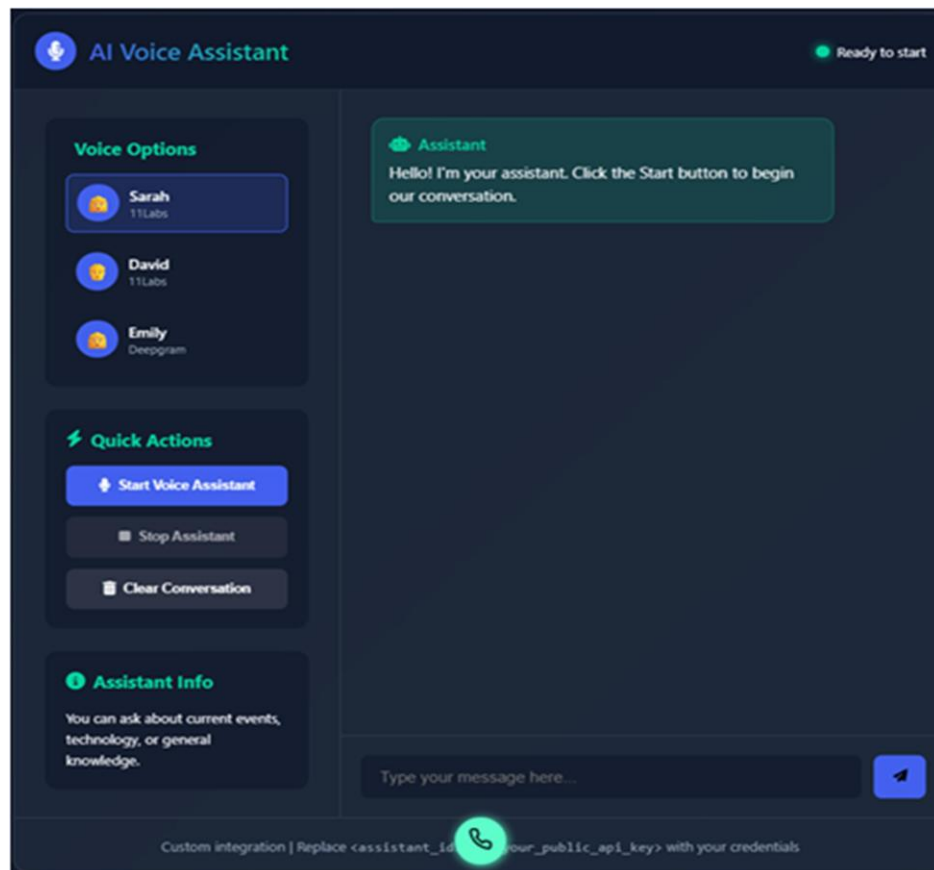
For other files refer below GitHub link
<https://github.com/darsh9679/Health-Voice-Assistant>

2.9 Experimental Results

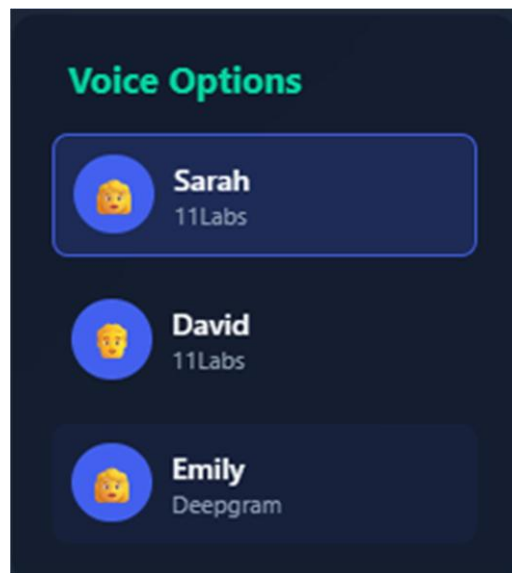
1) Chat with AI assistance:



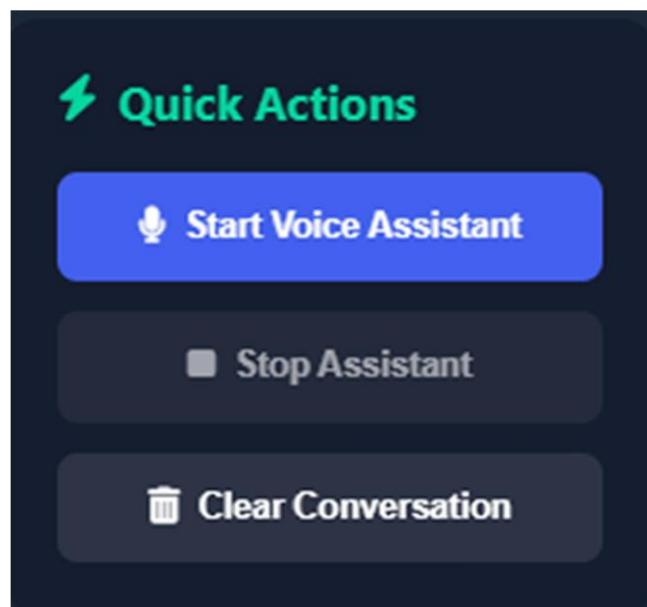
2) Landing Page:



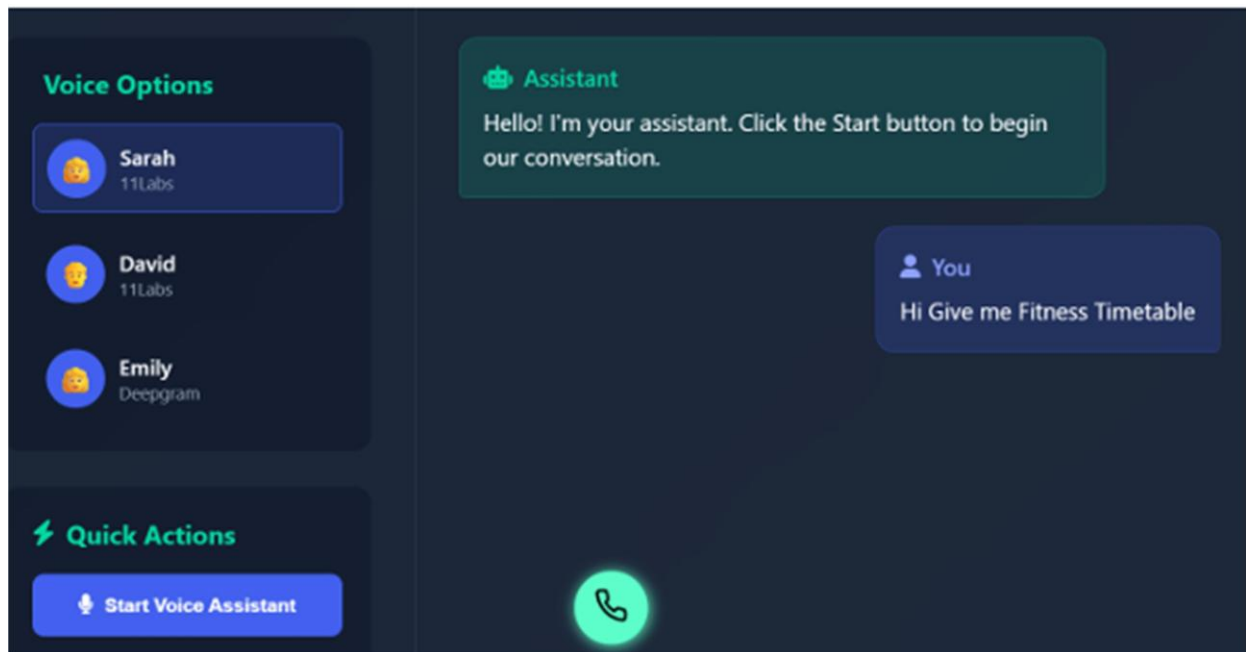
3) Different Voice Option based on Gender:



4) Quick Action to control AI assistance:



5) Chat with AI by enabling Microphone:



2.10 Conclusion and Future Work:

- This project successfully demonstrates the creation of a custom, web-based client for a sophisticated AI voice assistant. By leveraging the Vapi AI SDK and standard web technologies, we have built a solution that is more flexible and deeply integrable than common pre-built widgets. The event-driven architecture proves to be a robust model for handling the complexities of real-time voice communication.
- From a conceptual standpoint, the solution leverages Voice AI (VAPI) integrated with machine learning models to provide personalized workout recommendations, track progress, and adapt routines according to the user's fitness goals. By combining speech recognition, natural language processing, and cloud-based data storage, the system creates an interactive training experience that feels like a one on-one session with a real coach.

Future Work:

- Function Call Visualization: Implement UI components to visually represent when the assistant is executing a function call (e.g., fetching data from an API).
- Streaming Transcript: Enhance the transcript to show words as they are being transcribed in real-time, rather than only after a full utterance.
- Authentication: Integrate a user login system to store conversation history and user preferences.
- Componentization: Refactor the JavaScript code into modern framework components (e.g., React, Vue) for better scalability and maintainability in larger applications.

4 References:

- [1] Vapi Labs, Inc., "Vapi: The fastest way to build, test, and deploy voice AI applications," Vapi AI, 2024.
- [2] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, March 2023.
- [3] ElevenLabs, "Generative Voice AI," 2024.
- [4] A. Jennings, J. Uberti, and C. Holmer, "WebRTC 1.0: Real-Time Communication Between Browsers," W3C Recommendation, January 2021.
- [5] R. Pixley, H. Lie, C. Lilley, and D. Jacobs, "Cascading Style Sheets Level 3 (CSS3)," W3C, 2023.
- [6] D. Jurafsky and J. H. Martin, Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, 3rd ed. Prentice Hall, 2023.
- [7] Ecma International, "ECMAScript® 2023 Language Specification," Standard ECMA-262, 14th edition, June 2023.
- [8] M. Bodson, P. Adenot, and H. Thomsen, "Web Audio API," W3C Recommendation, June 2022.

Websites:

- <https://vapi.ai>
- <https://arxiv.org/abs/2303.08774>
- <https://elevenlabs.io>
- <https://www.w3.org/TR/webrtc/>
- <https://www.w3.org/Style/CSS/specs.en.html>
- <https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>
- <https://www.w3.org/TR/webaudio/>